

Frontend Interview — Complete Study Guide

Every question and scenario from your document, explained simply with examples.

How to use this: each item has a *plain-English answer* first, then a *tiny example* (an analogy, code, or both). Read it top to bottom once, then use the section list below to revise weak spots. Topics that repeat across companies are explained fully the first time and pointed to afterwards.

Sections

1. JavaScript Fundamentals
 2. Execution & Async Behaviour
 3. Data & References
 4. Browser & DOM
 5. Network & Side Effects
 6. Performance & Architecture
 7. React + Performance + Accessibility Interview
 8. Build & Bundling
 9. 25 Senior JavaScript Scenarios
 10. Preventing UI Flickering (React Lead)
 11. CORS & Web Performance
 12. Coforge React Round 1
 13. Senior React — Rendering Model
 14. JioHotstar SDE II (System Design + Machine Coding)
 15. useState vs useReducer
 16. Deloitte — Remaining Topics
-

1. JavaScript Fundamentals

1.1 `var`, `let`, `const` + block/function scope

- `var` is *function-scoped* — it ignores `{ }` blocks and belongs to the whole function.
- `let` and `const` are *block-scoped* — they only live inside the nearest `{ }`.
- `const` means you can't *reassign* the variable (but you can still mutate an object it points to).

```
function test() {  
  if (true) {  
    var a = 1; // leaks out of the if-block  
    let b = 2; // stays inside the if-block  
  }  
  console.log(a); // 1 ✓  
  console.log(b); // ReferenceError ✗  
}
```

Analogy: `var` is a note stuck on the office wall (everyone in the office sees it). `let` / `const` is a note on your desk (only visible in that room).

1.2 Hoisting & Temporal Dead Zone (TDZ)

Hoisting = JavaScript "lifts" declarations to the top of their scope before running code.

- `var` is hoisted and set to `undefined`.
- `let` / `const` are hoisted too, but you *cannot use them* until the line that declares them. That gap is the **Temporal Dead Zone**.

```
console.log(x); // undefined (var was hoisted)  
var x = 5;  
  
console.log(y); // ReferenceError - y is in the TDZ  
let y = 5;
```

Why it matters: it explains "why is my variable `undefined`?" bugs and "Cannot access before initialization" errors.

1.3 Closures & lexical environment

A **closure** is a function that *remembers the variables from where it was created*, even after that outer function has finished.

```
function counter() {  
  let count = 0;  
  return function () {  
    count++;  
    return count;  
  };  
}  
  
const next = counter();  
next(); // 1  
next(); // 2 - the inner function still remembers `count`
```

Analogy: the inner function carries a backpack of the variables it grew up with. **Lexical environment** just means "scope is decided by *where code is written*, not where it's called."

1.4 `this`, `call` / `apply` / `bind`, arrow functions

`this` = *who is calling the function* (decided at call-time for normal functions).

- `call` — call now, pass arguments one by one.
- `apply` — call now, pass arguments as an array.
- `bind` — don't call now, return a *new function* with `this` locked in.

```
const person = { name: "Raj" };  
function greet(greeting) { return `${greeting}, ${this.name}`; }  
  
greet.call(person, "Hi"); // "Hi, Raj"  
greet.apply(person, ["Hello"]); // "Hello, Raj"  
const bound = greet.bind(person);  
bound("Hey"); // "Hey, Raj"
```

Arrow functions have no `this` of their own — they borrow it from the surrounding scope. That's why they're great inside

callbacks:

```
const obj = {
  items: [1, 2],
  show() {
    this.items.forEach(() => console.log(this.items)); // `this` still = obj ✓
  }
};
```

1.5 Prototype chain & inheritance

Every object has a hidden link (`__proto__`) to another object it "inherits" from. When you access a property, JS walks *up this chain* until it finds it.

```
const animal = { eats: true };
const dog = Object.create(animal); // dog's prototype is animal
dog.barks = true;

dog.barks; // true (own property)
dog.eats;  // true (found on animal via the chain)
```

Analogy: ask your own drawer first; if it's empty, ask your parent's drawer, then grandparent's, and so on.

1.6 ES6 classes vs prototypes (what is syntax sugar)

`class` is **syntax sugar** — a cleaner way to write the exact same prototype mechanism. Under the hood there are no "real" classes like in Java; it's still prototypes.

```
class Dog {
  bark() { return "woof"; }
}
// Is the same as:
function Dog2() {}
Dog2.prototype.bark = function () { return "woof"; };
```

What is NOT just sugar: `extends` with built-ins (like `Array`), `super`, private fields (`#x`), and the fact that class bodies run in strict mode — these add genuinely new behaviour.

2. Execution & Async Behaviour

2.1 Event loop: call stack, task queue, microtask queue

JavaScript runs on **one thread**. The **event loop** is the traffic controller that decides what runs next.

- **Call stack** — where code runs right now (one thing at a time).
- **Microtask queue** — Promise callbacks (`.then`, `await`). *Higher priority*.
- **Task (macrotask) queue** — `setTimeout`, `setInterval`, clicks. *Lower priority*.

Rule: run all synchronous code → **empty the entire microtask queue** → run *one* macrotask → repeat.

```
console.log("1");
setTimeout(() => console.log("2"), 0); // macrotask
Promise.resolve().then(() => console.log("3")); // microtask
console.log("4");
```

// Output: 1, 4, 3, 2

Analogy: microtasks are VIP guests (served before anyone else); macrotasks wait in the normal line.

2.2 Promises & chaining

A **Promise** is a placeholder for a value that arrives later. `.then` transforms it; returning a value inside `.then` passes it to the next `.then`.

```
fetch("/user")
  .then(res => res.json()) // step 1
  .then(user => user.name) // step 2 (gets result of step 1)
  .then(name => console.log(name))
  .catch(err => console.log("failed", err)); // catches any step
```

2.3 async / await patterns (error handling, sequencing)

`async/await` is a cleaner way to write promises — it *looks* synchronous.

```
async function loadUser() {
  try {
    const res = await fetch("/user");
    const user = await res.json();
    return user;
  } catch (err) {
    console.log("failed", err); // one place catches everything
  }
}
```

Sequencing tip — run in parallel when tasks don't depend on each other:

```
// ✗ Slow: waits for A, then B
const a = await getA();
const b = await getB();

// ✔ Fast: both run together
const [a, b] = await Promise.all([getA(), getB()]);
```

2.4 Stale closures in async code

A **stale closure** = your async callback captured an *old* value and doesn't see the latest one.

```
let count = 0;
function start() {
  setTimeout(() => console.log(count), 1000); // captures count = 0
  count = 99;
}
start(); // prints 0 after 1s, not 99 — it "froze" the old value
```

In React this is the classic bug: a `setInterval` inside `useEffect` keeps logging the *initial* state. Fix with a `ref` or a functional state update (`setState(prev => prev + 1)`).

3. Data & References

3.1 Equality & coercion (== vs ===, truthy/falsy)

- `===` compares value *and* type (no surprises). **Always prefer this.**
- `==` converts types first, causing weird results.

```
0 == "";           // true (both become falsy)
0 == "0";          // true
null == undefined; // true
1 === "1";         // false ✓ (different types)
```

Falsy values (only 8): `false`, `0`, `-0`, `""`, `null`, `undefined`, `NaN`, `0n`. Everything else is truthy (including `"0"`, `[]`, and `{}`).

3.2 Deep vs shallow copy

- **Shallow copy** — copies the top level only; nested objects are *shared*.
- **Deep copy** — copies everything, fully independent.

```
const original = { name: "Raj", address: { city: "Chennai" } };

const shallow = { ...original };
shallow.address.city = "Delhi";
original.address.city; // "Delhi" ✗ (shared nested object!)

const deep = structuredClone(original); // modern deep copy
deep.address.city = "Mumbai";
original.address.city; // unaffected ✓
```

3.3 Object & array references

Objects/arrays are stored **by reference**. Two variables can point to the *same* box.

```
const a = [1, 2];
const b = a;           // same box, not a copy
b.push(3);
a; // [1, 2, 3] — a changed too!

const c = [1, 2];
const d = [1, 2];
c === d; // false — different boxes, even with same contents
```

Why interviewers love this: it explains "I changed one thing and something else broke" bugs, and why React state must be replaced (new reference) not mutated.

3.4 `map`, `filter`, `reduce`, `find`, `some`, `every`

Array helpers, in plain words:

```
const nums = [1, 2, 3, 4];

nums.map(n => n * 2);           // [2,4,6,8] transform each
nums.filter(n => n % 2 === 0); // [2,4] keep matches
nums.reduce((sum, n) => sum + n, 0); // 10 boil down to one value
nums.find(n => n > 2);           // 3 first match (or undefined)
nums.some(n => n > 3);           // true is ANY true?
nums.every(n => n > 0);          // true are ALL true?
```

Memory hook: `map` = change all, `filter` = pick some, `reduce` = combine into one, `find` = first hit, `some` = at least one, `every` = all of them.

4. Browser & DOM

4.1 DOM events & event delegation

Event delegation = instead of adding a listener to every child, add *one* listener to the parent and check which child was clicked. Works because events "bubble up."

```
// ✗ 100 listeners for 100 items
// ✓ 1 listener on the parent
document.querySelector("ul").addEventListener("click", (e) => {
  if (e.target.tagName === "LI") {
    console.log("Clicked:", e.target.textContent);
  }
});
```

Bonus: it also works for items added *later*, which individual listeners can't do.

4.2 Reflow vs repaint

- **Reflow (layout)** — the browser recalculates *positions and sizes*. Expensive. Triggered by changing width, height, adding/removing elements, reading `offsetHeight`.
- **Repaint** — the browser only *redraws pixels* (e.g. changing `color` or `background`). Cheaper.

Reflow always causes a repaint; a repaint doesn't cause a reflow.

```
el.style.width = "200px"; // reflow (geometry changed) → then repaint
el.style.color = "red";   // repaint only (no layout change)
```

Tip: batch DOM changes to avoid repeated reflows (change a CSS class once instead of many inline styles).

4.3 Debouncing vs throttling

Both limit how often a function runs — the difference is *when*.

- **Debounce** — wait until the user *stops*, then run once. (Search box.)
- **Throttle** — run at most once every X ms, no matter how often it fires. (Scroll / resize.)

```
// Debounce: fire only after 300ms of silence
function debounce(fn, delay) {
  let timer;
  return (...args) => {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

// Throttle: fire at most once per 200ms
function throttle(fn, delay) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= delay) { last = now; fn(...args); }
  };
}
```

Analogy: debounce = elevator waits until people stop entering, then moves. Throttle = bus leaves every 10 minutes regardless of how many arrive.

4.4 requestAnimationFrame (when and why)

`requestAnimationFrame` runs your code *right before the next screen repaint* (~60 times/sec), so animations stay smooth and pause automatically on hidden tabs.

```
function move() {
  box.style.left = (parseFloat(box.style.left || 0) + 1) + "px";
  requestAnimationFrame(move); // schedule the next frame
}
requestAnimationFrame(move);
```

Use it instead of `setInterval` for animation — it syncs with the display and doesn't waste work when nothing is visible.

5. Network & Side Effects

5.1 Fetch API

`fetch` makes network requests and returns a Promise. **Gotcha:** it does *not* reject on 404/500 — you must check `res.ok` yourself.

```
async function getData() {
  const res = await fetch("/api/data");
  if (!res.ok) throw new Error(`HTTP ${res.status}`); // must check!
  return res.json();
}
```

5.2 AbortController (why cancellation matters)

Lets you *cancel* a fetch — vital when a user types fast or leaves a page, so old requests don't overwrite new results.

```
const controller = new AbortController();
fetch("/api/search?q=raj", { signal: controller.signal })
  .catch(err => { if (err.name === "AbortError") console.log("cancelled"); });

controller.abort(); // cancel it
```

Real use: in a search box, abort the previous request before firing the new one.

5.3 CORS basics (what FE controls vs what it doesn't)

CORS is a browser security rule: a page on `siteA.com` can't freely call `siteB.com` unless the server allows it.

- **Frontend controls:** the request (headers, method, credentials).
- **Backend controls:** whether it's allowed — via `Access-Control-Allow-Origin` etc.

You *cannot* fix a CORS error purely in frontend code — the server (or a proxy) must send the right headers. (More in Section 11.)

5.4 Web storage vs cookies (when to use what)

	Sent to server?	Size	Use for
Cookies	Yes, every request	~4KB	Auth tokens the server needs (esp. <code>httpOnly</code>)
localStorage	No	~5-10MB	Data that persists across sessions (theme, drafts)
sessionStorage	No	~5-10MB	Data for one tab/session (wizard progress)

```
localStorage.setItem("theme", "dark"); // stays after closing browser
sessionStorage.setItem("step", "2"); // cleared when tab closes
```

Rule of thumb: tokens the backend reads → cookies (`httpOnly` for security). Client-only data → local/session storage.

6. Performance & Architecture

6.1 Pure functions & testable code

A **pure function** always returns the same output for the same input and changes nothing outside itself. Easy to test, cache, and reason about.

```
// Pure ✓  
const add = (a, b) => a + b;  
  
// Impure ✗ (depends on & changes outside state)  
let total = 0;  
const addToTotal = (n) => { total += n; };
```

6.2 When memoization helps vs hurts

Memoization = cache a function's result so you don't recompute it.

- **Helps** when a computation is *expensive* and inputs repeat.
- **Hurts** when the computation is *cheap* — the caching overhead and memory cost outweigh the savings.

```
const cache = {};  
function slowSquare(n) {  
  if (cache[n]) return cache[n]; // reuse  
  return (cache[n] = n * n); // compute + store  
}
```

Rule: measure first. Don't memoize a `+1`.

6.3 Web Workers (what problems they solve)

JavaScript is single-threaded, so a heavy calculation *freezes the UI*. A **Web Worker** runs that work on a *separate thread*, keeping the page responsive.

```
// main.js  
const worker = new Worker("worker.js");  
worker.postMessage(bigData);  
worker.onmessage = (e) => console.log("result:", e.data);  
  
// worker.js  
onmessage = (e) => {  
  const result = heavyCalculation(e.data);  
  postMessage(result);  
};
```

Use for: large data processing, image manipulation, parsing big files — anything that would otherwise make the page hang. Workers can't touch the DOM directly.

7. React + Performance + Accessibility Interview

7.1 How would you optimize slow initial loading time?

Attack it in layers:

1. **Code splitting** — ship only what the first screen needs (`React.lazy`).
2. **Lazy load** images and below-the-fold components.
3. **Compress & minify** JS/CSS; enable gzip/Brotli on the server.
4. **Cache** with hashed filenames + a CDN.
5. **Reduce bundle** — remove unused libraries, tree shake.
6. **Server-side render / stream** the first paint if SEO or speed matters.

One-liner for the interview: "Send less, send it compressed, and send it split."

7.2 Common causes of slow initial loading

- One giant JS bundle (no splitting).
- Large unoptimized images.
- Too many blocking `<script>` / `<link>` tags.
- Heavy third-party scripts (analytics, chat widgets).
- Fetching data *after* render instead of prefetching.
- No caching / no CDN.

7.3 Atomic Design Pattern

A way to structure UI by building from **small to big** (Brad Frost's model):

- **Atoms** — button, input, label.
- **Molecules** — a search bar (input + button).
- **Organisms** — a header (logo + nav + search bar).
- **Templates** — page layout with placeholders.
- **Pages** — template filled with real content.

Benefit: reusable, consistent components. **Analogy:** Lego bricks → small builds → full model.

7.4 What is accessibility (a11y) and why it matters

Accessibility = building so *everyone* can use your app, including people using screen readers, keyboards only, or with low vision.

- Use semantic HTML (`<button>` , not a clickable `<div>`).
- Add `alt` text to images.
- Ensure keyboard navigation (Tab, Enter).
- Use `aria-*` labels where needed; keep good colour contrast.

Why it matters: it's the right thing to do, it's often a legal requirement, it improves SEO, and it helps *all* users (e.g. captions help in noisy places).

7.5 Controlled vs uncontrolled components

- **Controlled** — React state is the single source of truth (`value` + `onChange`). You always know the current value.
- **Uncontrolled** — the DOM keeps the value; you read it with a `ref` when needed.

```
// Controlled
const [name, setName] = useState("");
<input value={name} onChange={e => setName(e.target.value)} />

// Uncontrolled
const ref = useRef();
<input ref={ref} />
// read later: ref.current.value
```

7.6 Huge form with many fields — which approach?

Use **uncontrolled inputs (refs)** or a library like **React Hook Form**. Reason: a *controlled* huge form re-renders on *every* *keystroke* in *every* *field*, which lags. Uncontrolled/RHF only reads values on submit, so it stays fast.

Interview line: "Controlled is great for a few fields with live validation; for large forms I go uncontrolled or React Hook Form to avoid re-rendering on every keystroke."

7.7 Live coding — form: log input, then reset

```
function MyForm() {
  const [value, setValue] = useState("");

  const handleSubmit = () => {
    console.log(value); // log it
    setValue("");      // reset it
  };

  return (
    <div>
      <input
        value={value}
        onChange={(e) => setValue(e.target.value)}
        placeholder="Type here"
      />
      <button onClick={handleSubmit}>Submit</button>
    </div>
  );
}
```

(In React artifacts avoid `<form>` tags — use `onClick` on the button, as above.)

7.8 Purpose of the `placeholder` attribute

`placeholder` shows *hint* text inside an empty input that disappears when the user types. It is a **hint, not a label** — it vanishes, so it shouldn't be the only description of the field.

```
<input placeholder="Enter your email" />
```

7.9 Indicate a required field *without* a placeholder

Use a real `<label>` plus the `required` attribute (and often a visible `*`):

```
<label for="email">Email *</label>
<input id="email" type="email" required aria-required="true" />
```

This is accessible (screen readers announce it) and doesn't disappear like placeholder text.

7.10 Multiple inputs, form centered on the page

Use flexbox to center:

```
<div style={{
  display: "flex",
  flexDirection: "column",
  alignItems: "center",
  justifyContent: "center",
  minHeight: "100vh",
  gap: "10px"
}}>
  <input placeholder="Name" />
  <input placeholder="Email" />
  <input placeholder="Phone" />
  <button>Submit</button>
</div>
```

`alignItems: center` centers horizontally, `justifyContent: center` + `minHeight: 100vh` centers vertically.

8. Build & Bundling

8.1 What happens when you run `npm run build`?

It creates an **optimized production version** of your app. Roughly:

1. **Transpiles** modern JS/JSX → browser-compatible JS (Babel/SWC).
2. **Bundles** many files into a few (Webpack/Vite).
3. **Tree shakes** unused code.
4. **Minifies** (removes spaces, shortens names).
5. Adds **hashed filenames** for caching.

Output lands in a `build/` or `dist/` folder — static files ready to deploy.

8.2 How does Vite work internally, and why is it faster than CRA?

- In **dev**, Vite serves your code as **native ES modules** over the browser — no bundling upfront. The browser requests only the modules it needs, so startup is near-instant. It uses **esbuild** (written in Go) for pre-bundling dependencies, which is very fast.
- **CRA (Create React App)** uses Webpack, which **bundles the whole app before** the dev server starts — slow on big projects.
- In **production**, Vite bundles with **Rollup** for an optimized output.

One-liner: "CRA bundles everything before you see anything; Vite serves modules on demand and uses esbuild, so it's much faster to start and hot-reload."

8.3 Tree shaking

Tree shaking removes code you *import but never use*, shrinking the bundle. It works because ES modules (`import / export`) are *statically analysable*.

```
// utils.js
export function used() {}
export function unused() {}

// app.js
import { used } from './utils'; // `unused` is dropped from the build
```

8.4 Code splitting — `React.lazy()` and `Suspense`

Code splitting breaks the bundle into chunks loaded *on demand*, so the first load is smaller.

```
const Dashboard = React.lazy(() => import('./Dashboard'));

<Suspense fallback=<p>Loading...</p>>
  <Dashboard />
</Suspense>
```

`React.lazy` loads the component only when it's rendered; `Suspense` shows a fallback while it downloads. **Use it for** heavy pages, routes, or modals not needed immediately.

8.5 Bundling vs transpiling vs minification

Three different jobs:

- **Transpiling** — convert new syntax to older syntax (JSX/ES2023 → ES5). *Tool: Babel.*
- **Bundling** — combine many files into fewer. *Tool: Webpack/Vite/Rollup.*
- **Minification** — remove whitespace/comments, shorten names to reduce size. *Tool: Terser.*

Analogy: transpile = translate to a language everyone understands; bundle = staple pages together; minify = shrink the font and remove blank space.

8.6 Source mapping — and why not always in production

A **source map** links minified production code back to your original readable code, so error stack traces make sense while debugging.

Why hide in production: source maps expose your original source code to anyone. Common practice: generate them but

restrict access (upload to an error-tracking tool like Sentry rather than serving them publicly).

8.7 Analyze and reduce a large JS bundle

Steps: 1. Analyze with a tool (`webpack-bundle-analyzer` , `vite-plugin-visualizer` , `source-map-explorer`) to see what's big. 2. Replace heavy libraries with lighter ones (e.g. `date-fns` instead of full Moment). 3. Import only what you use (`import debounce from "lodash/debounce"`). 4. Code split routes and heavy components. 5. Tree shake and drop dead dependencies.

8.8 Cache busting & hashed filenames

Browsers cache files by name. If you change code but keep the name `main.js` , users see the *old cached* version. **Hashed filenames** (`main.a1b2c3.js`) change when content changes, forcing a fresh download — that's **cache busting**.

Benefit: you can cache aggressively (fast) *and* still ship updates reliably.

8.9 Configure different environments (dev / staging / prod)

Use **environment variables** in `.env` files:

```
.env.development → VITE_API_URL=http://localhost:8000
.env.staging     → VITE_API_URL=https://staging.api.com
.env.production  → VITE_API_URL=https://api.com
```

```
const api = import.meta.env.VITE_API_URL; // Vite
// (or process.env.REACT_APP_API_URL in CRA)
```

The build tool injects the right value per environment. (*This is exactly the `EXPO_PUBLIC_API_URL` pattern you hit on MSMECapitalLink — wrong env value → app points at the wrong backend.*)

8.10 Optimization strategies before deploying

A quick checklist: - Production build (minified, tree shaken). - Code splitting + lazy loading. - Compress images; use modern formats (WebP). - Enable gzip/Brotli + CDN + long-cache with hashed names. - Remove `console.log` and source-map exposure. - Audit with **Lighthouse**; fix largest issues (LCP, bundle size).

9. 25 Senior JavaScript Scenarios

These test *how you think when things break*. Short answer + code for each.

9.1 Works in dev, throws in production after minification

Minifiers rename variables. The usual culprit: code that relies on **function/class names or string keys** that got renamed, or a dependency that breaks when mangled. **Debug:** enable **source maps** to map the error back, check for name-dependent logic, and test a production build locally (`npm run build && serve`).

9.2 Memory leak in a long-running SPA

Common causes: listeners/timers never cleaned up, detached DOM nodes still referenced, growing caches, closures holding big data. **Find it:** Chrome DevTools → **Memory tab** → take heap snapshots over time and compare what keeps growing.

Fix: clean up in `useEffect` return, clear intervals, remove listeners.

```
useEffect(() => {  
  const id = setInterval(tick, 1000);  
  return () => clearInterval(id); // cleanup prevents leak  
}, []);
```

9.3 Run 10 API calls, max 3 at a time (concurrency limit)

```
async function limit(tasks, max) {  
  const results = [];  
  const running = new Set();  
  for (const [i, task] of tasks.entries()) {  
    const p = Promise.resolve(task()).then(r => (results[i] = r));  
    running.add(p);  
    p.finally(() => running.delete(p));  
    if (running.size >= max) await Promise.race(running); // wait for a slot  
  }  
  await Promise.all(running);  
  return results;  
}
```

9.4 A third-party script blocks the main thread

Load it `async` or `defer` so it doesn't block parsing; or move heavy work into a **Web Worker**; or load it *lazily* after the page is interactive.

```
<script src="analytics.js" defer></script>
```

9.5 NaN appears in a UI calculation

NaN means a number operation got a non-number (e.g. `Number(undefined)` or `"abc" * 2`). **Trace:** log inputs, add guards.

Fix: validate/convert before math.

```
const price = Number(input) || 0; // fall back to 0 if not a number
```

9.6 Recursive function → stack overflow on large input

Convert recursion to a **loop** (iteration) so it doesn't pile up stack frames.

```
// ✖ overflows  
function sum(n) { return n === 0 ? 0 : n + sum(n - 1); }  
// ✔ iterative  
function sum(n) { let t = 0; for (let i = 1; i <= n; i++) t += i; return t; }
```

9.7 Implement `Promise.all` from scratch

```
function promiseAll(promises) {
  return new Promise((resolve, reject) => {
    const results = [];
    let done = 0;
    if (promises.length === 0) return resolve(results);
    promises.forEach((p, i) => {
      Promise.resolve(p).then(val => {
        results[i] = val;
        if (++done === promises.length) resolve(results);
      }, reject); // any rejection rejects the whole thing
    });
  });
}
```

9.8 Throttle a scroll handler to once / 200ms (no libraries)

```
function throttle(fn, delay) {
  let last = 0;
  return (...args) => {
    const now = Date.now();
    if (now - last >= delay) { last = now; fn(...args); }
  };
}
window.addEventListener("scroll", throttle(onScroll, 200));
```

9.9 A button click sometimes fires twice

Causes: listener attached twice, event bubbling to two handlers, a re-render re-binding, or a fast double-tap. **Fixes:** attach once, `stopPropagation` if needed, disable the button while processing, or debounce the handler.

```
button.disabled = true; // prevent double submit
await doWork();
button.disabled = false;
```

9.10 Publish/subscribe event bus (vanilla JS)

```
function createEventBus() {
  const map = {};
  return {
    on(evt, cb) { (map[evt] ||= []).push(cb); },
    off(evt, cb) { map[evt] = (map[evt] || []).filter(f => f !== cb); },
    emit(evt, ...args) { (map[evt] || []).forEach(cb => cb(...args)); }
  };
}
```

9.11 Loop registers listeners but all share the same value

Classic `var` closure bug — every listener sees the *final* value. **Fix:** use `let` (new binding per iteration).

```
// ✗ all log 3
for (var i = 0; i < 3; i++) btns[i].onclick = () => console.log(i);
// ✓ logs 0,1,2
for (let i = 0; i < 3; i++) btns[i].onclick = () => console.log(i);
```

9.12 Safely parse & validate JSON from an untrusted API

Wrap `JSON.parse` in `try/catch`, then **validate the shape** before using it (manually or with a schema library like Zod).

```
function safeParse(text) {
  try {
    const data = JSON.parse(text);
    if (typeof data?.name !== "string") throw new Error("bad shape");
    return data;
  } catch { return null; }
}
```

9.13 Deeply compare two nested objects

```
function deepEqual(a, b) {
  if (a === b) return true;
  if (typeof a !== "object" || typeof b !== "object" || !a || !b) return false;
  const ka = Object.keys(a), kb = Object.keys(b);
  if (ka.length !== kb.length) return false;
  return ka.every(k => deepEqual(a[k], b[k]));
}
```

9.14 Date calculations wrong across timezones

The bug: mixing local time and UTC. **Fix:** store/compute in **UTC** (or ISO strings) and only convert to local for display. For anything serious, use a library (`date-fns-tz` , Luxon).

```
new Date().toISOString(); // UTC, safe to store
```

9.15 Lazy evaluation using closures/generators

Compute values only when needed (saves work, enables infinite sequences).

```
function* naturals() {
  let n = 1;
  while (true) yield n++; // produced only when asked
}
const g = naturals();
g.next().value; // 1
g.next().value; // 2
```

9.16 Synchronous `<script>` blocks rendering for 3s (can't change server)

Add `async` or `defer` to the tag, or inject the script dynamically after load so it stops blocking the parser.

```
const s = document.createElement("script");
s.src = "heavy.js";
s.defer = true;
document.body.appendChild(s);
```

9.17 Task queue — async tasks run strictly one at a time

Chain each new task onto the previous promise.

```
class TaskQueue {
  constructor() { this.chain = Promise.resolve(); }
  add(task) {
    this.chain = this.chain.then(() => task());
    return this.chain;
  }
}
```

9.18 "Closures always cause memory leaks" — agree?

No. Closures only leak if they *keep large data alive longer than needed* (e.g. a listener you never remove). Used normally they're released by garbage collection. It's a real concern only when a long-lived reference holds onto heavy state.

9.19 Detect circular references when serializing to JSON

`JSON.stringify` throws on cycles. Track seen objects with a `WeakSet` .

```
function safeStringify(obj) {
  const seen = new WeakSet();
  return JSON.stringify(obj, (key, val) => {
    if (val && typeof val === "object") {
      if (seen.has(val)) return "[Circular]";
      seen.add(val);
    }
    return val;
  });
}
```

9.20 Ensure a function runs exactly once, even called concurrently

For async, **cache the promise** so concurrent callers share one run.

```
function onceAsync(fn) {
  let promise;
  return (...args) => (promise ||= fn(...args));
}
```

9.21 Long `forEach` freezes the tab — chunk it (no Web Worker)

Process a slice, then yield to the browser with `setTimeout`, and repeat.

```
function processInChunks(items, doItem, size = 100) {
  let i = 0;
  (function next() {
    const end = Math.min(i + size, items.length);
    for (; i < end; i++) doItem(items[i]);
    if (i < items.length) setTimeout(next, 0); // let UI breathe
  })();
}
```

9.22 Fetch retry — up to 3 retries, 1s apart

```
async function fetchRetry(url, options, retries = 3, delay = 1000) {
  for (let attempt = 0; attempt <= retries; attempt++) {
    try {
      const res = await fetch(url, options);
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return res;
    } catch (err) {
      if (attempt === retries) throw err;
      await new Promise(r => setTimeout(r, delay));
    }
  }
}
```

9.23 Track which properties of an object are accessed (Proxy)

```
function trackAccess(obj) {
  const accessed = new Set();
  const proxy = new Proxy(obj, {
    get(target, prop) { accessed.add(prop); return target[prop]; }
  });
  return { proxy, accessed };
}
```

9.24 Function called with wrong context in a callback (don't change caller)

Lock `this` with `bind` or wrap in an **arrow function**.

```
setTimeout(this.handleClick.bind(this), 100);
// or
setTimeout(() => this.handleClick(), 100);
```

9.25 Simple observable — subscribe / unsubscribe / emit

```
class Observable {
  constructor() { this.subs = []; }
  subscribe(fn) {
    this.subs.push(fn);
    return () => { this.subs = this.subs.filter(s => s !== fn); }; // unsubscribe
  }
  emit(data) { this.subs.forEach(fn => fn(data)); }
}
```


10. Preventing UI Flickering (React Lead)

10.1 What causes flickering, and how to fix it

Flickering = the "empty → loading → jump" flash users see. Causes and fixes:

Cause	Fix
Data fetched <i>after</i> first render	Show skeleton loaders ; cache with React Query/SWR
State read from <code>localStorage</code> inside <code>useEffect</code>	Initialize state correctly <i>before</i> first render
Components remounting due to unstable <code>key</code>	Use stable keys (an id, not the array index or <code>Math.random()</code>)
Images loading without reserved size	Set <code>width</code> / <code>height</code> so layout doesn't jump
Unnecessary re-renders	<code>React.memo</code> , <code>useMemo</code> , <code>useCallback</code>
CSS loading after render (FOUC)	Inline critical CSS; load styles early

```
// ✓ initialize from storage BEFORE first render (no flicker)
const [theme, setTheme] = useState(() => localStorage.getItem("theme") || "light");
```

10.2 Follow-up: why `useLayoutEffect` instead of `useEffect` to prevent flicker?

- `useEffect` runs **after** the browser paints. If you change layout there, the user briefly sees the *old* version → flicker.
- `useLayoutEffect` runs **after DOM changes but before paint**. You can measure/adjust the DOM and the user only ever sees the *final* result.

Real example: a tooltip that must position itself based on its measured size. With `useEffect` it flashes in the wrong spot then jumps; with `useLayoutEffect` it's already correct on first paint.

```
useLayoutEffect(() => {
  const { height } = ref.current.getBoundingClientRect();
  ref.current.style.top = `-${height}px`; // adjusted before paint → no flash
}, []);
```

Caution: `useLayoutEffect` blocks painting, so use it *only* when you must read/change layout before paint — otherwise `useEffect` is better for performance.

11. CORS & Web Performance

11.1 Why does the browser fire an extra request? What is it?

It's a **CORS preflight** — an automatic **OPTIONS** request the browser sends *before* your real request, to ask the server "Am I allowed to do this?"

It happens for "**non-simple**" requests: methods like **PUT** / **DELETE**, custom headers (e.g. **Authorization**), or a **Content-Type** of **application/json**. So your **POST /api/tickets** becomes two round-trips: **OPTIONS** then **POST**.

11.2 How to optimize it in production

Tell the browser to **cache the preflight result** so it doesn't re-ask every time:

```
Access-Control-Max-Age: 7200
```

This caches the "yes, you're allowed" answer for 2 hours, so repeated calls skip the **OPTIONS** step → fewer requests, faster pages. (Since browsers allow only ~6 connections per domain, cutting these extra calls frees up the queue.) Other wins: keep requests "simple" where possible, and use HTTP/2 (Section 14).

12. Coforge React Round 1

12.1 Reconciliation & how the Virtual DOM minimizes DOM work

The **Virtual DOM** is a lightweight JS copy of the real DOM. On each update React: 1. Builds a new virtual tree. 2. **Diff**s it against the previous one (reconciliation). 3. Updates **only the real DOM nodes that actually changed**.

Real DOM changes are slow; doing the minimum keeps things fast. **Keys** help React match list items correctly during the diff.

12.2 `useMemo` VS `useCallback` VS `React.memo`

- `useMemo` — caches a *computed value*.
- `useCallback` — caches a *function* (so its reference stays stable).
- `React.memo` — caches a *component* (skips re-render if props didn't change).

```
const total = useMemo(() => expensiveCalc(items), [items]); // value
const onClick = useCallback(() => doThing(id), [id]); // function
const Child = React.memo(function Child(props) { /* ... */ }); // component
```

When: use `React.memo` on a pure child that re-renders too often; pass it a stable `onClick` via `useCallback`; use `useMemo` for genuinely expensive calculations.

12.3 `useEffect` VS `useLayoutEffect` VS `useInsertionEffect`

Order they run in after a render:

1. `useInsertionEffect` — runs *first*, before DOM changes. Meant for **CSS-in-JS libraries** to inject styles. App developers rarely use it.
2. `useLayoutEffect` — after DOM changes, *before paint*. For measuring/adjusting layout (Section 10.2).
3. `useEffect` — *after paint*. For data fetching, subscriptions, most side effects. **Your default**.

12.4 Role-based authentication & authorization in React

- **Authentication** = who are you (login → token).
- **Authorization** = what are you allowed to do (roles/permissions).

Store the user's role, then guard routes/UI:

```
function ProtectedRoute({ allowedRoles, children }) {
  const { role } = useAuth();
  if (!role) return <Navigate to="/login" />;
  if (!allowedRoles.includes(role)) return <Navigate to="/forbidden" />;
  return children;
}
// <ProtectedRoute allowedRoles={['admin']}><AdminPage /></ProtectedRoute>
```

Important: frontend checks are for UX only — the **backend must enforce** permissions too, since client code can be bypassed.

12.5 Infinite scrolling with server-side pagination + caching

Load more data as the user nears the bottom.

- Detect the bottom with **IntersectionObserver**.
- Fetch the next **page** from the server (`?page=2`).
- **Cache** pages (React Query's `useInfiniteQuery` handles this well).

```
const observer = new IntersectionObserver([entry] => {
  if (entry.isIntersecting) loadNextPage();
});
observer.observe(bottomRef.current);
```

12.6 Redux Toolkit internals — "direct mutation" but immutable?

Redux Toolkit uses **Immer** under the hood. You *write* mutating code on a special **draft**, and Immer produces a brand-new immutable state behind the scenes.

```
// Looks like mutation, but Immer makes a new immutable copy:
reducers: {
  addItem: (state, action) => { state.items.push(action.payload); }
}
```

So you get simple syntax *and* immutability. RTK also bundles Thunk, DevTools, and less boilerplate.

12.7 Shared state across multiple micro-frontends

Options, simplest to strongest: - **Custom events** / **window event bus** for loose messaging. - **A shared store** exposed by a container app (or Module Federation shared singletons). - **URL / storage** for simple shared values. - **Backend as source of truth** for real data.

Principle: keep shared state *minimal* — micro-frontends should stay as independent as possible; share events or a thin store, not everything.

12.8 Why React batches updates (and the Event Loop's role)

React **batches** multiple state updates into *one* re-render for performance. In React 18+ this happens even inside promises/timeouts (automatic batching). The updates are queued and flushed together, aligning with the event-loop tick, so you don't get a re-render per `setState`.

```
setCount(c => c + 1);
setName("Raj");
// → one re-render, not two
```

12.9 Polyfill for `Promise.allSettled()`

`allSettled` waits for *all* promises and reports each outcome (never rejects).

```
function allSettled(promises) {
  return Promise.all(
    promises.map(p =>
      Promise.resolve(p).then(
        value => ({ status: "fulfilled", value }),
        reason => ({ status: "rejected", reason })
      )
    )
  );
}
```

12.10 Output question — async / event loop

```
console.log("A");
async function foo() {
  console.log("B");
  await Promise.resolve();
  console.log("C");
  await Promise.resolve();
  console.log("D");
}
foo();
Promise.resolve().then(() => console.log("E"));
setTimeout(() => console.log("F"), 0);
console.log("G");
```

Output: A B G C E D F

Why: 1. Sync runs first: A, then `foo()` logs B and hits `await` (pauses), then G. 2. Microtasks queued in order: C (foo's continuation), then E. 3. Run C → it hits another `await`, queuing D after E. Run E. Then run D. 4. Finally the macrotask F (`setTimeout` always last).

12.11 Output question — recursive closure counter

```
function createCounter() {  
  let count = 0;  
  function increment() {  
    count++;  
    if (count < 3) increment(); // recurses  
    return count;  
  }  
  return increment;  
}  
const fn = createCounter();  
console.log(fn()); // 3  
console.log(fn()); // 4
```

Why: First `fn()` — count goes 1 → recurse → 2 → recurse → 3, condition fails, returns `count` which is now **3**. `count` persists in the closure, so second `fn()` — count goes to **4** (`4 < 3` is false, returns 4).

13. Senior React — Rendering Model

13.1 What triggers a re-render?

Four things: 1. **State changes** (`setState`). 2. **Parent re-renders** (children re-render by default). 3. **Context value changes** (all consumers re-render). 4. **New props**.

13.2 Why doesn't React update the *entire* DOM?

Because after each render React does **diffing + reconciliation** on the Virtual DOM, then touches **only the real nodes that changed**. Updating the whole DOM would be slow; targeted updates are fast.

13.3 Why do unnecessary re-renders happen?

- Parent re-renders frequently.
- New **object/array/function references** created each render (they look "different" to React).
- Inline functions passed as props.
- Context updates.
- Poorly structured state (one big state object changing too often).

13.4 When should you use memoization?

Only after measuring. Use `React.memo` / `useMemo` / `useCallback` when profiling shows *expensive computations* or *proven unnecessary re-renders*. Blind memoization adds complexity and memory cost for no gain.

13.5 How would you optimize a slow React app?

The senior answer is a *process*: 1. **Profile** with React DevTools Profiler. 2. Find unnecessary renders. 3. **Split** big components. 4. Apply `React.memo` **selectively**. 5. Use `useMemo` / `useCallback` where they actually help. 6. **Lazy load** heavy components. 7. **Reduce bundle** size.

Key line: "Most people list React features; the senior move is to *measure first*, then fix the real bottleneck."

14. JioHotstar SDE II (System Design + Machine Coding)

14.1 Round 1 topic recap

All covered above — quick pointers: - **Promises / output questions** → Sections 2 & 12.10. - **Event loop, micro vs macrotasks** → Section 2.1. - **Critical Rendering Path** → the browser's steps from code to pixels: HTML → **DOM**, CSS → **CSSOM**, combine → **Render Tree** → **Layout** → **Paint**. Fewer/smaller blocking resources = faster first paint. - **Bundle optimization, code splitting, lazy loading** → Section 8. - **Reflow vs repaint** → Section 4.2. - **HTTP/1.1 vs HTTP/2** → see below.

HTTP/1.1 vs HTTP/2: - HTTP/1.1: one request at a time per connection → browsers open ~6 connections; suffers "head-of-line blocking." - HTTP/2: **multiplexing** (many requests over one connection at once), **header compression**, and server push. Result: faster loads, fewer connections needed. *(This is also why the CORS preflight caching in Section 11 matters less on HTTP/2 but still helps.)*

14.2 System Design — scalable video player

Think in these buckets (say them out loud in the interview):

- **API / data:** endpoints for video metadata, streaming manifest (HLS/DASH), and analytics.
- **Adaptive streaming:** serve multiple quality levels; switch based on the user's bandwidth (HLS/DASH) so playback doesn't buffer.
- **Component architecture:** separate `Player`, `Controls`, `ProgressBar`, `Captions`, `QualitySelector` — reusable and independently testable.
- **Performance:** lazy load the player, preload the next segment, use a CDN for video delivery.
- **Accessibility:** captions, keyboard controls (space = play/pause), ARIA labels.
- **Theming & reuse:** props/config so multiple teams can restyle it.
- **Error handling:** retry failed segments, graceful fallback, reconnection logic.
- **Trade-offs:** more buffering = smoother but higher data; lower latency = more requests. Name the trade-off you'd choose and why.

Interview approach: clarify requirements → sketch components → cover streaming + performance + a11y → discuss trade-offs. They want your *thinking*, not a perfect design.

14.3 Machine coding round

This tests building a **working, production-minded** React component under time pressure. What they grade: - **State design** (clean, minimal). - **Edge cases** (empty, loading, error states). - **Accessibility** (keyboard, labels). - **Error handling & caching** where relevant. - **Communication** (explain as you build).

Practice order (from your document, and it's good advice): Counter → Toggle → List → Phone Book → Todo → Context Toggle → Todo with reducer → Star Rating → Directory Tree → Tic Tac Toe → Autocomplete (offline → online) → Transfer List → Dynamic Forms → Shopping Cart → Async list with tests.

Tip: speed comes from repetition. Build 8–10 of these until the boilerplate is muscle memory, then focus your interview energy on edge cases and clear communication.

15. useState vs useReducer

Use `useState` for simple, independent state: counters, toggles, single inputs.

Use `useReducer` when state is *complex*: multiple related fields, updates that depend on previous state, or multi-step logic (forms, carts, dashboards, API state).

```
// useState - simple
const [count, setCount] = useState(0);

// useReducer - complex, related state with clear rules
const initial = { loading: false, data: [], error: null };
function reducer(state, action) {
  switch (action.type) {
    case "FETCH_START": return { ...state, loading: true };
    case "FETCH_SUCCESS": return { ...state, loading: false, data: action.payload };
    case "FETCH_ERROR": return { ...state, loading: false, error: action.error };
    default: return state;
  }
}
const [state, dispatch] = useReducer(reducer, initial);
dispatch({ type: "FETCH_START" });
```

Analogy: `useState` is a sticky note (quick, simple). `useReducer` is a checklist with rules (best when many actions change the same state). It's not about writing less code — it's about staying readable as the app grows.

16. Deloitte — Remaining Topics

Most Deloitte topics are covered above. Pointers + the few new ones:

- **Why React.js** → component reuse, Virtual DOM efficiency, huge ecosystem, one-way data flow makes apps predictable.
- **Session vs Local Storage** → Section 5.4 (session = one tab; local = persists).
- **Context API & use cases** → share global data (theme, auth, language) without "prop drilling." Best for low-frequency updates; heavy updates → a state library.
- **Auth strategies** → Section 12.4.
- **addEventListener / event handling** → Section 4.1.
- **useLayoutEffect vs useEffect** → Section 12.3.
- **Debouncing & throttling** → Section 4.3.
- **Event loop** → Section 2.1.
- **useState & useEffect** → Sections 15 & 2.4.
- **Redux & performance** → Section 12.6 (+ use selectors, avoid over-subscribing).
- **Lazy loading & code splitting** → Section 8.4.
- **Reconciliation algorithm** → Section 12.1.
- **React Fiber & Concurrent Rendering** → *Fiber* is React's rewritten engine that can **pause, resume, and prioritize** rendering work (instead of one uninterruptible pass). *Concurrent rendering* builds on it so React can keep the UI responsive during big updates (e.g. `useTransition`).
- **CSR vs SSR vs SSG** →
 - **CSR** (Client-Side Rendering): browser builds the page with JS. Fast navigation, weaker initial SEO.
 - **SSR** (Server-Side Rendering): server sends ready HTML per request. Good SEO, fresh data, more server load.
 - **SSG** (Static Site Generation): HTML built at *build time*. Fastest, great for content that rarely changes.
- **useRef vs useState** → `useRef` stores a value that **survives re-renders but does NOT trigger one** (good for DOM refs, timers, previous values). `useState` triggers a re-render.
- **React.memo** → Section 12.2.
- **Custom Hooks** → reusable logic extracted into a `useSomething()` function. `javascript function useToggle(initial = false) { const [on, setOn] = useState(initial); const toggle = () => setOn(o => !o); return [on, toggle]; }`
- **Preventing unnecessary re-renders** → Section 13.3.
- **Role-Based Access Control (RBAC)** → Section 12.4.
- **Redux Toolkit vs Context API vs TanStack Query** →
 - **Context** = share simple global state (theme/auth).
 - **Redux Toolkit** = complex *client* state with strict structure.
 - **TanStack (React) Query** = **server** state — fetching, caching, refetching, sync. (Often you use Query for server data + Context/RTK for UI state.)
- **TypeScript Generics** → reusable types that work with any type. `typescript function first<T>(arr: T[]): T { return arr[0]; } first<number>([1, 2]); // T = number`
- **Utility Types (Partial , Pick , Omit , Record)** → `typescript type User = { id: number; name: string; age: number }; Partial<User>; // all fields optional Pick<User, "id" | "name">; // only id & name Omit<User, "age">; // everything except age Record<string, number>; // { [key: string]: number }`
- **Microtasks vs Macrotasks** → Section 2.1.
- **API Caching, Retries & Background Sync** → cache responses (React Query), retry failed calls (Section 9.22), and re-sync in the background when the tab regains focus / connection returns.
- **Scalable Notification System Design** → central store for notifications, real-time delivery via **WebSockets** (or polling), a queue for ordering, read/unread state, and a UI that batches updates to avoid flicker.
- **Real-time dashboard (live data) — system design** →
 - **WebSockets vs Polling**: WebSockets for true real-time (persistent 2-way connection); polling (or long-polling) is simpler but heavier. Choose based on update frequency.
 - **State management**: a single store for live data; update efficiently.
 - **Caching**: keep last-known data so the UI isn't empty on reconnect.
 - **Component optimization**: memoize charts/widgets so only changed ones re-render.
 - **Error handling & reconnection**: detect drops, auto-reconnect with backoff, show a "reconnecting" state.

Final tips for the room

- When asked "how does X work," explain the **why** and **what breaks if misused** — that's what separates senior candidates.
- For anything performance-related, say "**measure first**" (Lighthouse / React DevTools) before optimizing.
- For system design, **clarify requirements → sketch components → cover performance + accessibility + error handling → name trade-offs**.
- For live coding, **talk as you type**, handle empty/loading/error states, and keep it clean.

Good luck, Raj. You've got this.